WEEK 3

Question 1
Correct
Marked out of 1.00
Flag question

Write a program to find the minimum and second minimum elements with in the elements of one dimensional array.

Constraints:

- 1 <= N <= 10^3
- 1 <= Elements of the array <= 10^6

Instruction: To run your custom test cases strictly map your input and output layout with the visible test cases.

Hints
Let us assume that first element it self as the **minimum**, **second_minimum** and then compare both with all the other elements.
If any one found as **minimum** then change the value of the minimum, otherwise compare it with **second_minimum** and exchange it when necessary condition is satisfied.

For example:

Input	Result
4 65 32 85 96	Min element = 32 Second min element = 65

Answer: (penalty regime: 0 %)

```
#include <stdio.h>
#include <limits.h>

int main()
{
    int n;
    if(scanf("%d", &n) != 1)
    return 0;

    if(n<2)
    {
        return 0;
    }
    int arr[n];
    for(int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    int min = INT_MAX;
    int second_min = INT_MAX;

```

	Input	Expected	Got	
✔	4 65 32 85 96	Min element = 32 Second min element = 65	Min element = 32 Second min element = 65	✔

Passed all tests! ✔

➔ Thus, the program is executed successfully

Question 2
Correct
Marked out of 1.00
Flag question

Write a program to read a student n subjects marks in an array and find the total, average of the marks.

For example, if the user gives the **input** as:
3

Next, the program should print the messages one by one on the console.
if the user gives the **input** as:
75
80
85

then the program should **print** the result as:

```
The total marks = 240
The average marks = 80.000000
```

Hints
marks are integers, **total** is also an integer but **average** is a float value, so typecast it.

For example:

Input	Result
5 45 65 55 75 85	The total marks = 325 The average marks = 65.000000
4 36 45 38 56	The total marks = 175 The average marks = 43.750000

Answer: (penalty regime: 0 %)

```
#include <stdio.h>

int main()
{
    int n, total = 0;
    float average;

    if(scanf("%d",&n) != 1 || n <= 0)
    {
        return 0;
    }

    int marks[n];

    for(int i = 0; i<n; i++)
    {
        scanf("%d", &marks[i]);
        total += marks[i];
    }

    average =(float)total/n;

```

	Input	Expected	Got	
✔	5 45 65 55 75 85	The total marks = 325 The average marks = 65.000000	The total marks = 325 The average marks = 65.000000	✔
✔	4 36 45 38 56	The total marks = 175 The average marks = 43.750000	The total marks = 175 The average marks = 43.750000	✔

Passed all tests! ✔

➔ Thus, the program is executed successfully

Question 3
Correct
Marked out of 1.00
Flag question

The below sample code finds the addition of two matrices.

In the main() function read a two two-dimensional array of elements and then find the addition of two matrices.

The logic is

- First checks the **row sizes** and **column sizes** of two two-dimensional arrays are equal or not.
- If the sizes are not equal then print "Addition is not possible" and stop the process.
- If the sizes are equal then use **two for loops** to add each corresponding elements of two matrices and finally print the result.

  Fill in the missing code so that it produces the desired output.

For example:

Input	Result
2 2 1 2 3 4 2 2 4 5 6 7	The given matrix-1 is 1 2 3 4 The given matrix-2 is 4 5 6 7 Addition of two matrices is 5 7 9 11

Answer: (penalty regime: 0 %)

Reset answer

```c
#include <stdio.h>
int main()
{
    int i, j, m, n, p, q;
    int a[5][5], b[5][5], c[5][5];
    scanf("%d %d", &m, &n);
    for (i = 0; i<m; i++ )
    { // Complete the code in for
        for (j = 0; j<n;j++)
        { // Complete the code in for
            scanf("%d", &a[i][j]);
        }
    }
    scanf("%d %d", &p, &q);
    for (i = 0; i<p; i++ )
    { // Complete the code in for
        for (j = 0; j<q; j++ )
        { // Complete the code in for
            scanf("%d", &b[i][j]);
        }
    }
    printf("The given matrix-1 is\n");
```

	Input	Expected	Got	
✔	2 2 1 2 3 4 2 2 4 5 6 7	The given matrix-1 is 1 2 3 4 The given matrix-2 is 4 5 6 7 Addition of two matrices is 5 7 9 11	The given matrix-1 is 1 2 3 4 The given matrix-2 is 4 5 6 7 Addition of two matrices is 5 7 9 11	✔

Passed all tests! ✔

➔ Thus, the program is executed successfully

Question 4
Correct
Marked out of 1.00
Flag question

The below sample code finds the multiplication of two matrices.

In the main() function read a two two-dimensional array of elements and then find the multiplication of two matrices.

The logic is

- First check the **column size** of **first matrix** and **row size** of **second matrix** are equal or not.
- If the sizes are not equal then print "multiplication is not possible" and stop the process.
- If the sizes are equal then use **three for loops** to multiply the elements of two matrices and finally print the result.

Fill in the missing code so that it produces the desired result.
For example:

Input	Result
2 3 1 2 3 5 7 9 3 2 3 2 1 5 6 7	The given matrix-1 is 1 2 3 5 7 9 The given matrix-2 is 3 2 1 5 6 7 Multiplication of two matrices is 23 33 76 108

Answer: (penalty regime: 0 %)

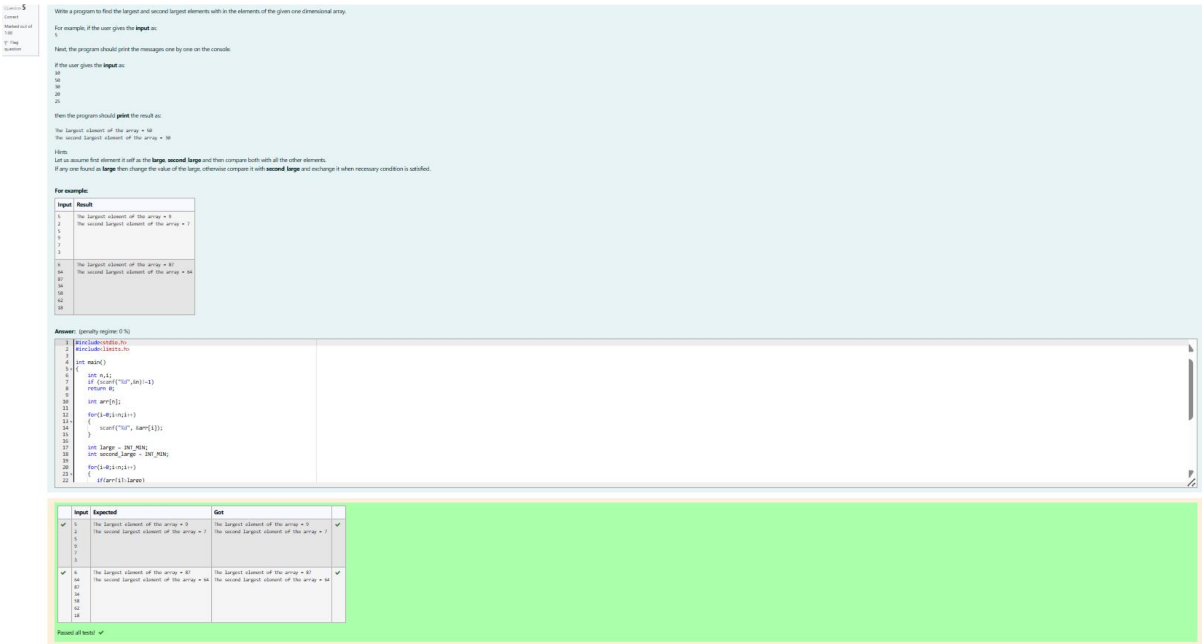
Reset answer

```c
#include <stdio.h>
int main()
{
    int i, j, k, m, n, p, q;
    int a[5][5], b[5][5], c[5][5];
    scanf("%d %d", &m, &n);
    for (i=0;i<m;i++ )
    { // Complete the code in for
        for (j=0;j<n;j++ )
        { // Complete the code in for
            scanf("%d", &a[i][j]);
        }
    }
    scanf("%d %d", &p, &q);
    for (i=0;i<p;i++ )
    { // Complete the code in for
        for (j=0;j<q;j++ )
        { // Complete the code in for
            scanf("%d", &b[i][j]);
        }
    }
    printf("The given matrix-1 is\n");
```

	Input	Expected	Got	
✔	2 3 1 2 3 5 7 9 3 2 3 2 1 5 6 7	The given matrix-1 is 1 2 3 5 7 9 The given matrix-2 is 3 2 1 5 6 7 Multiplication of two matrices is 23 33 76 108	The given matrix-1 is 1 2 3 5 7 9 The given matrix-2 is 3 2 1 5 6 7 Multiplication of two matrices is 23 33 76 108	✔

Passed all tests! ✔

➔ Thus, the program is executed successfully



Question 5
Correct
Marked out of 1.00
Flag question

Write a program to find the largest and second largest elements with in the elements of the given one dimensional array.

For example, if the user gives the **input** as:
5

Next, the program should print the messages one by one on the console.

if the user gives the **input** as:
10
50
30
20
25

then the program should **print** the result as:

```
The largest element of the array = 50
The second largest element of the array = 30
```

Hints
Let us assume first element it self as the **large**, **second large** and then compare both with all the other elements.
If any one found as **large** then change the value of the large, otherwise compare it with **second large** and exchange it when necessary condition is satisfied.

For example:

Input	Result
5 2 5 9 7 3	The largest element of the array = 9 The second largest element of the array = 7
6 64 87 34 58 62 18	The largest element of the array = 87 The second largest element of the array = 64

Answer: (penalty regime: 0 %)

```
#include<stdio.h>
#include<limits.h>

int main()
{
    int n,i;
    if (scanf("%d",&n)!=1)
    return 0;

    int arr[n];

    for(i=0;i<n;i++)
    {
        scanf("%d", &arr[i]);
    }

    int large = INT_MIN;
    int second_large = INT_MIN;

    for(i=0;i<n;i++)
    {
        if(arr[i]>large)
```

	Input	Expected	Got	
✔	5 2 5 9 7 3	The largest element of the array = 9 The second largest element of the array = 7	The largest element of the array = 9 The second largest element of the array = 7	✔
✔	6 64 87 34 58 62 18	The largest element of the array = 87 The second largest element of the array = 64	The largest element of the array = 87 The second largest element of the array = 64	✔

Passed all tests! ✔

➔Thus, the program is executed successfully

Question 1
Correct
Marked out of 1.00
Flag question

Fill in the missing code in the below sample code which counts the number of vowels, consonants, digits and spaces are presented in a given string.

Initially, the variables vowels, consonants, digits and spaces are initialized to 0.

Iterate the string from the **first** character to **last** character to find all vowels, consonants, digits and spaces.

When a vowel character is found, vowel variable is incremented by 1. Similarly, consonants, digits and spaces are incremented when these characters are found in the string.

Finally, the count is displayed on the screen.

For example:

Input	Result
kohli hits 100 in every cricket match!	Vowels = 9 Consonants = 19 Digits = 3 White spaces = 6

Answer: (penalty regime: 0 %)

Reset answer

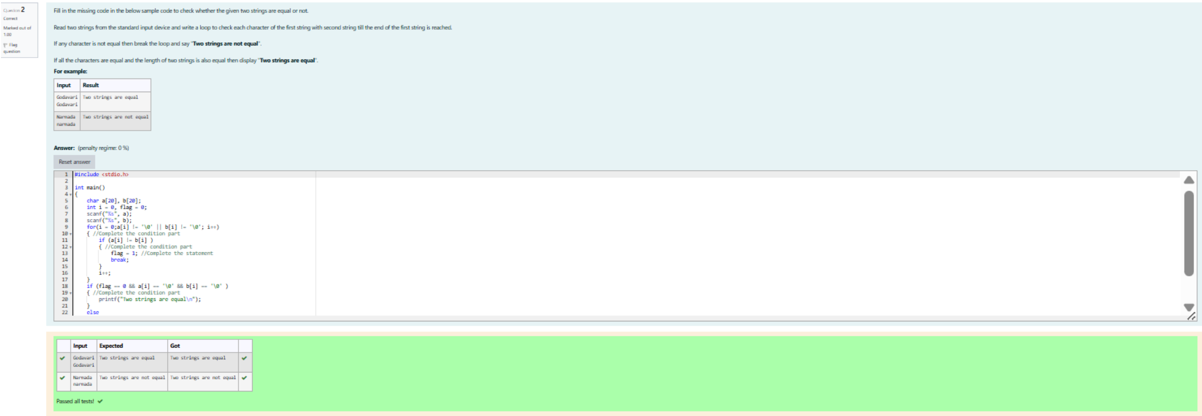
```
#include <stdio.h>

int main()
{
    char line[100];
    int i, vowels = 0, consonants = 0, digits = 0, spaces = 0;
    fgets(line, sizeof(line), stdin);
    for (i = 0; line[i] != '\0'; i++)
    { // Complete the code in for
        if (line[i]=='a'||line[i]=='e'||line[i]=='i'||line[i]=='o'||line[i]=='u'||
            line[i]=='A'||line[i]=='E'||line[i]=='I'||line[i]=='O'||line[i]=='U')
        { // Write the condition part
            ++vowels;
        }
        else if ((line[i]>='a' && line[i]<='z')||(line[i]>='A' && line[i]<='Z'))
        { // Write the condition part
            ++consonants;
        }
        else if (line[i]>='0' && line[i]<='9')
        { // Write the condition part
            ++digits;
        }
```

	Input	Expected	Got	
✔	kohli hits 100 in every cricket match!	Vowels = 9 Consonants = 19 Digits = 3 White spaces = 6	Vowels = 9 Consonants = 19 Digits = 3 White spaces = 6	✔

Passed all tests! ✔

➔Thus, the program is executed successfully



Question 2
Correct
Marked out of 1.00
Flag question

Fill in the missing code in the below sample code to check whether the given two strings are equal or not.

Read two strings from the standard input device and write a loop to check each character of the first string with second string till the end of the first string is reached.

If any character is not equal then break the loop and say "**Two strings are not equal**".

If all the characters are equal and the length of two strings is also equal then display "**Two strings are equal**".

For example:

Input	Result
Godavari Godavari	Two strings are equal
Narmada narmada	Two strings are not equal

Answer: (penalty regime: 0 %)

Reset answer

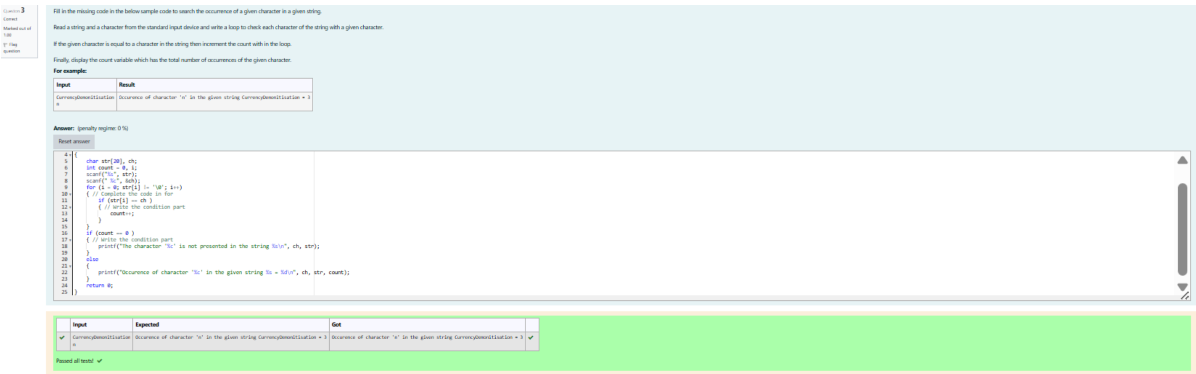
```c
#include <stdio.h>

int main()
{
    char a[20], b[20];
    int i = 0, flag = 0;
    scanf("%s", a);
    scanf("%s", b);
    for(i = 0;a[i] != '\0' || b[i] != '\0'; i++)
    { //Complete the condition part
        if (a[i] != b[i] )
        { //Complete the condition part
            flag = 1; //Complete the statement
            break;
        }
        i++;
    }
    if (flag == 0 && a[i] == '\0' && b[i] == '\0' )
    { //Complete the condition part
        printf("Two strings are equal\n");
    }
    else
```

	Input	Expected	Got	
✔	Godavari Godavari	Two strings are equal	Two strings are equal	✔
✔	Narmada narmada	Two strings are not equal	Two strings are not equal	✔

Passed all tests! ✔

➔Thus, the program is executed successfully



Question 3
Correct
Marked out of 1.00
Flag question

Fill in the missing code in the below sample code to search the occurrence of a given character in a given string.

Read a string and a character from the standard input device and write a loop to check each character of the string with a given character.

If the given character is equal to a character in the string then increment the count with in the loop.

Finally, display the count variable which has the total number of occurrences of the given character.

For example:

Input	Result
CurrencyDemonitisation n	Occurence of character 'n' in the given string CurrencyDemonitisation = 3

Answer: (penalty regime: 0 %)

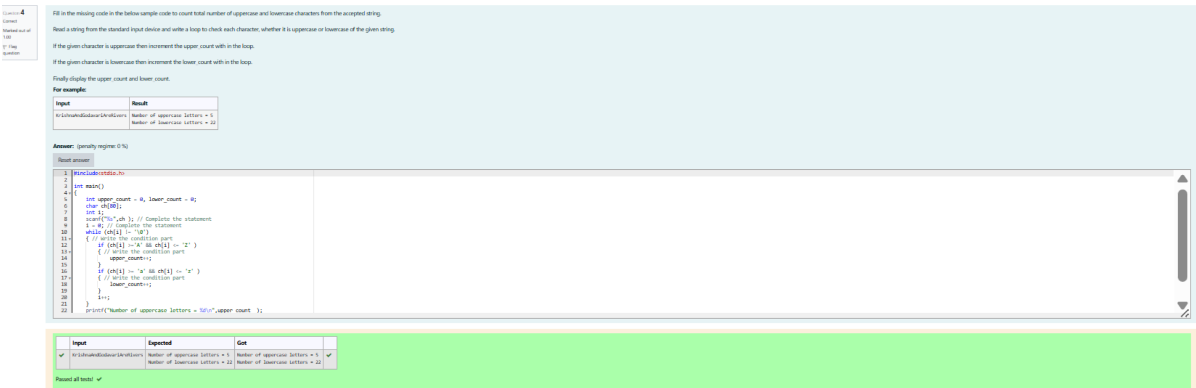
Reset answer

```c
{
    char str[20], ch;
    int count = 0, i;
    scanf("%s", str);
    scanf(" %c", &ch);
    for (i = 0; str[i] != '\0'; i++)
    { // Complete the code in for
        if (str[i] == ch )
        { // Write the condition part
            count++;
        }
    }
    if (count == 0 )
    { // Write the condition part
        printf("The character '%c' is not presented in the string %s\n", ch, str);
    }
    else
    {
        printf("Occurence of character '%c' in the given string %s = %d\n", ch, str, count);
    }
    return 0;
}
```

	Input	Expected	Got	
✔	CurrencyDemonitisation n	Occurence of character 'n' in the given string CurrencyDemonitisation = 3	Occurence of character 'n' in the given string CurrencyDemonitisation = 3	✔

Passed all tests! ✔

➔Thus, the program is executed successfully



Question 4
Correct
Marked out of 1.00
Flag question

Fill in the missing code in the below sample code to count total number of uppercase and lowercase characters from the accepted string.

Read a string from the standard input device and write a loop to check each character, whether it is uppercase or lowercase of the given string.

If the given character is uppercase then increment the upper_count with in the loop.

If the given character is lowercase then increment the lower_count with in the loop.

Finally display the upper_count and lower_count.

For example:

Input	Result
KrishnaAndGodavariAreRivers	Number of uppercase letters = 5 Number of lowercase Letters = 22

Answer: (penalty regime: 0 %)

Reset answer

```c
#include<stdio.h>

int main()
{
    int upper_count = 0, lower_count = 0;
    char ch[80];
    int i;
    scanf("%s",ch ); // Complete the statement
    i = 0; // Complete the statement
    while (ch[i] != '\0')
    { // Write the condition part
        if (ch[i] >='A' && ch[i] <= 'Z' )
        { // Write the condition part
            upper_count++;
        }
        if (ch[i] >= 'a' && ch[i] <= 'z' )
        { // Write the condition part
            lower_count++;
        }
        i++;
    }
    printf("Number of uppercase letters = %d\n",upper_count  );
```

	Input	Expected	Got	
✔	KrishnaAndGodavariAreRivers	Number of uppercase letters = 5 Number of lowercase Letters = 22	Number of uppercase letters = 5 Number of lowercase Letters = 22	✔

Passed all tests! ✔

➔ Thus, the program is executed successfully

Question 5
Correct
Marked out of 1.00
Flag question

Fill in the missing code in the below sample code to reverse the given string.

Hints
Step:1 Read a string from the standard input device.
Step:2 Write a loop to find the length of the string.
Step:3 Write another loop to interchange the characters from first to last of the string.
Step:4 Finally display the reverse of a string.

For example:

Input	Result
Software	The reverse of a given string : erawtfoS

Answer: (penalty regime: 0 %)

Reset answer

```
#include<stdio.h>

int main()
{
    char ch[80], temp;
    int i, j;
    scanf("%s", ch);
    i = j = 0;
    while (ch[j] != '\0' )
    { // Write the condition part
        j++;
    }
    j--;
    while (i < j )
    { // Write the condition part
        temp = ch[i]; // Complete the statement
        ch[i] = ch[j]; // Complete the statement
        ch[j] = temp; // Complete the statement
        i++;
        j--;
    }
    printf("The reverse of a given string : %s\n", ch);
```

	Input	Expected	Got	
✔	Software	The reverse of a given string : erawtfoS	The reverse of a given string : erawtfoS	✔

Passed all tests! ✔

➔ Thus, the program is executed successfully

Question 6
Correct
Marked out of 1.00
Flag question

Fill in the missing code in the below sample code to check whether the given string is a palindrome or not.

Read a string from the standard input device and write a loop to check the characters of the given string with the reverse string.

If all the characters are equal then display "**The given string is a palindrome**", otherwise display "**The given string is not a palindrome**".

For example:

Input	Result
12321	The given string 12321 is a palindrome
amaravathi	The given string amaravathi is not a palindrome

Answer: (penalty regime: 0 %)

Reset answer

```
#include <stdio.h>

int main()
{
    char ch[80];
    int i, j, length, flag = 0;
    scanf("%s",ch ); // Complete the statement
    length = 0;
    while (ch[length] != '\0' )
    { //Write the condition part
        length++;
    }
    i = 0;
    j = length - 1;
    while (i < j )
    { // Complete the code in for
        if (ch[i] != ch[j] )
        { // Write the condition part
            flag = 1;
            break;
        }
        i++;
```

	Input	Expected	Got	
✔	12321	The given string 12321 is a palindrome	The given string 12321 is a palindrome	✔
✔	amaravathi	The given string amaravathi is not a palindrome	The given string amaravathi is not a palindrome	✔

Passed all tests! ✔

➔ Thus, the program is executed successfully

Question 7
Correct
Marked out of 1.00
Flag question

Fill in the missing code in the below sample code which concatenates two given strings and store the result in another string.

Read two strings from the standard input device and write a loop to copy each character of the first string into third string till the end of the first string.

Write another loop to copy each character of the second string into third string till the end of second string.

Now place '\0' at the end of the third string.

Finally, display the third string.

For example:

Input	Result
Narendra Modi	NarendraModi

Answer: (penalty regime: 0 %)

Reset answer

```
#include <stdio.h>

int main()
{
    char a[20], b[20], c[20];
    int i, j;
    scanf("%s", a);
    scanf("%s", b);
    for (i = 0; a[i] != '\0'; i++  )
    { // Complete the code in for
        c[i] = a[i] ; //Complete the statement
    }
    for (j = 0; b[j] != '\0'; j++ )
    { // Complete the code in for
        c[i] = b[j] ; //Complete the statement
        i++;
    }
    c[i] = '\0' ; //Complete the statement
    printf("%s", c);
    return 0;
}
```

	Input	Expected	Got	
✔	Narendra Modi	NarendraModi	NarendraModi	✔

Passed all tests! ✔

➔Thus, the program is executed successfully

Question 8
Correct
Marked out of 1.00
Flag question

Fill in the missing code in the below sample code which copies a given string into another string.

Initially, read a string from the standard input device and write a loop to copy each character of given string into another string till the end of the string is reached.

Place '\0' at the end of the copied string.

Finally, the copied string is displayed on the screen.

For example:

Input	Result
GangaRiver	The copied string = GangaRiver

Answer: (penalty regime: 0 %)

Reset answer

```
#include <stdio.h>

int main()
{
    char str1[50], str2[50];
    int i;
    scanf("%s", str1);
    for (i = 0; str1[i] != '\0'; i++ )
    { //Complete the code in for
        str2[i] = str1[i];
    }
    str2[i] = '\0'; //Complete the statement
    printf("The copied string = %s\n", str2);
    return 0;
}
```

	Input	Expected	Got	
✔	GangaRiver	The copied string = GangaRiver	The copied string = GangaRiver	✔

Passed all tests! ✔

➔Thus, the program is executed successfully

Question 1
Correct
Marked out of 1.00
Flag question

The function strcat() is used to concatenate two strings into a single string.

The syntax of strcat() is strcat(string1, string2);.

where string1, string2 are two different strings. Here string2 is concatenated with string1, and the **concatenated string** is stored in string1.

In strcat() operation, **NULL character ('\0')** of string1 is **overwritten** by first character of string2 and **NULL character ('\0')** is appended (added) at the end of **new** string1 which is created after strcat() operation.

Fill the missing code in the below program to display the **concatenated** string using **strcat()** function.

For example:

Input	Result
REC Chennai	RECChennai

Answer: (penalty regime: 0 %)

Reset answer

```
#include <stdio.h>
#include <string.h>

int main()
{
    char str1[20], str2[20];
    scanf("%s", str1);
    scanf("%s", str2);
    strcat(str1,str2);
    printf("%s\n",str1 ); // Correct the code
    return 0;
}
```

	Input	Expected	Got	
✔	REC Chennai	RECChennai	RECChennai	✔

Passed all tests! ✔

➔Thus, the program is executed successfully

Question 2
Correct
Marked out of 1.00
Flag question

The function strcmp() is used for comparison of two strings and it always returns the numeric data. This function compares strings character by character using their ASCII values.

The syntax of strcmp() is variable_name = strcmp (string1, string2);.

Where string1, string2 are two strings and the variable is of **integer** datatype.

The comparison of two strings is dependent on the **alphabets (characters)** and not on the size (length) of the strings.

If the function strcmp() returns zero, both strings are **equal**.
If the function strcmp() returns a value which is less than zero, **string2** is higher than **string1** (because the ASCII value of first unmatched character of **string1** is less than the ASCII value of the corresponding character in **string2**)
If the function strcmp() returns a value which is greater than zero, **string1** is higher than **string2** (because the ASCII value of first unmatched character of **string1** is greater than the ASCII value of the corresponding character in **string2**)
Fill the missing code in the below program to compare two strings using **strcmp()** function.

For example:

Input	Result
NarendraModi narendramodi	The string narendramodi is higher than the string NarendraModi
Krishna Godavari	The string Krishna is higher than the string Godavari
REC REC	The given two strings are equal

Answer: (penalty regime: 0 %)

Reset answer

```c
#include <stdio.h>
#include <string.h>

int main()
{
    char a[20], b[20];

    scanf("%s", a);
    scanf("%s", b);
    //Compare two strings

    if (strcmp(a,b)==0 )
    { // Correct the code
        printf("The given two strings are equal\n");
    }
    else if (strcmp(a,b)>0 )
    { // Correct the code
        printf("The string %s is higher than the string %s\n", a, b);
    }
    else
    {
        printf("The string %s is higher than the string %s\n", b, a);
```

	Input	Expected	Got	
✔	NarendraModi narendramodi	The string narendramodi is higher than the string NarendraModi	The string narendramodi is higher than the string NarendraModi	✔
✔	Krishna Godavari	The string Krishna is higher than the string Godavari	The string Krishna is higher than the string Godavari	✔
✔	REC REC	The given two strings are equal	The given two strings are equal	✔

Passed all tests! ✔

➔ Thus, the program is executed successfully

Question 3
Correct
Marked out of 1.00
Flag question

The function strcpy() is used to **copy** one string into another string including the NULL character (terminator char '\0').

The syntax of strcpy( ) is strcpy(string1, string2);.

Where string1, string2 are two strings and the string2 is copied into string1. In this case the copied string is available in string1 and both strings contains the same data.

If the length of string1 is less than the length of string2 then entire string2 value will not be copied into string1.

For example, consider the length of string1 is **20** and the length of string2 is **30**. Then, only the first **20** characters from string2 will be copied into string1, the remaining **10** characters will not be copied and will be **truncated**.

Understand and retype the below code which demonstrates the usage of **strcpy()** function.

```c
#include <stdio.h>
#include <string.h>

int main()
{
  char str1[20], str2[20];
  scanf("%s", str2);
  strcpy(str1, str2);
  printf("The copied string = %s", str1);
  return 0;
}
```

For example:

Input	Result
Rose	The copied string = Rose

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <string.h>

int main()
{
    char str1[20], str2[20];
    scanf("%s", str2);
    strcpy(str1,str2);

    printf("The copied string = %s", str1);

    return 0;
}
```

	Input	Expected	Got	
✔	Rose	The copied string = Rose	The copied string = Rose	✔

Passed all tests! ✔

➔ Thus, the program is executed successfully

Question 4
Correct
Marked out of 1.00
Flag question

In **C** language, we have four types of string functions that are used for performing **string operations**. They are strlen(), strcpy(), strcat(), strcmp().

The function strlen() is used to find the **length** of the given string. This function returns only the **integer data** (or) **numeric data**.

The function strlen() counts the number of characters in a given string and returns the integer value.

It stops counting the character when **NULL** character is found. Because, **NULL** character indicates the end of the string in **C**.

The syntax of strlen() is integer_variable = strlen(string);.

Here string is a group of characters, strlen() function finds the **length** of the string and the **integer** value will be stored in the integer_variable.

The string.h header file supports all the string functions in **C** language.

Fill in the missing code in the below program to find the **length** of a string using **strlen()** function.

For example:

Input	Result
NarendraModi	The length of the string NarendraModi is 12

Answer: (penalty regime: 0 %)

Reset answer

```
#include <stdio.h>
#include <string.h>

int main()
{
    char ch[20];
    scanf("%s", ch);
    printf("The length of the string %s is %ld\n", ch,strlen(ch) ); //Correct the code
    return 0;
}
```

	Input	Expected	Got	
✔	NarendraModi	The length of the string NarendraModi is 12	The length of the string NarendraModi is 12	✔

Passed all tests! ✔

➔ Thus, the program is executed successfully